Pós·Unifor

LABORATÓRIO PRÁTICO · AULA DE 04/10

Metabase na EC2 com RDS PostgreSQL e ALB

Do zero até um painel de BI no ar: EC2 Amazon Linux com Docker, Metabase via docker-compose guardando tudo num RDS PostgreSQL free tier, e um Application Load Balancer publicando a interface.

DISCIPLINA
Computação em Nuvem
Pós-Graduação Lato Sensu

REGIÃO AWS
us-east-1
N. Virginia

DURAÇÃO ESTIMADA
~2h hands-on
+ limpeza

🛡️ 3 Security Groups · 🐘 RDS PostgreSQL free tier · 🖥️ EC2 Amazon Linux 2023
🐳 Docker + Compose · 📊 Metabase · ⚖️ Application Load Balancer · 🧪 Dados de exemplo

Prof. Bruno Vinícius Barroso Cavalcanti · AWS Solutions Architect Professional · CKA

ANTES DE COMEÇAR

0 Visão geral do laboratório

O que vamos construir, em que ordem e por quê

Neste laboratório você vai publicar o **Metabase** — uma ferramenta open source de BI para criar perguntas, gráficos e dashboards em cima de um banco de dados — numa **EC2 com Amazon Linux 2023**, rodando em **Docker** via **docker-compose**. O Metabase não vai guardar nada no disco da EC2: o banco interno dele (usuários, perguntas, dashboards) fica num **RDS PostgreSQL free tier**, que também hospeda um banco de exemplo (`loja`) com 20 mil vendas para você analisar. Na frente da EC2 fica um **Application Load Balancer**, que é o único caminho para acessar a interface pela Internet.

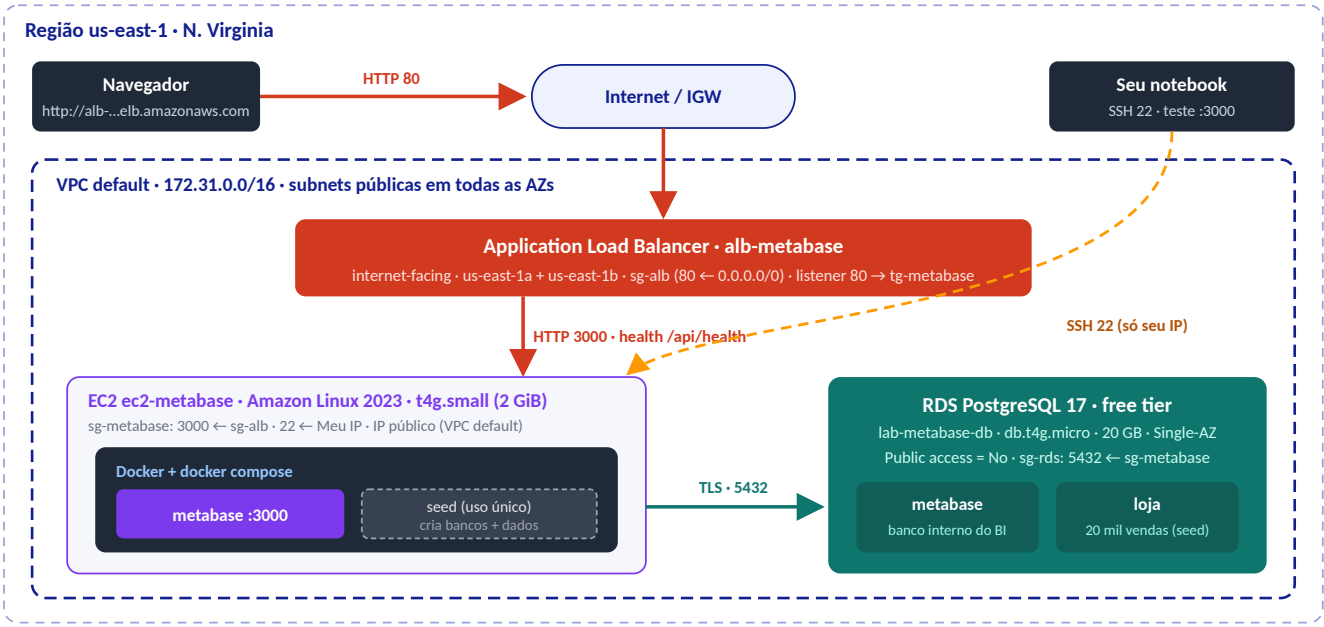


Figura 1 — Arquitetura do lab. Cada security group só aceita tráfego da camada anterior: Internet → ALB → EC2 → RDS.

Roteiro

I	**Security groups** — `sg-alb`, `sg-metabase` e `sg-rds` encadeados	10 min
II	**RDS PostgreSQL free tier** — template Free tier, privado, banco inicial `metabase`	15 min
III	**EC2 Amazon Linux 2023** — t4g.small, key pair e primeiro SSH	10 min
IV	**Docker + Compose** — instalar, criar `.env` com as credenciais do RDS, seed e subir o Metabase	20 min
V	**Application Load Balancer** — target group :3000, health check e listener :80	15 min
VI	**Metabase** — setup inicial, conectar o banco `loja` e montar um dashboard	25 min
VII	**Provas e desafios** — onde os dados moram, recriar o container sem perder nada	10 min

Tabela de nomes e valores (use exatamente estes)

Recurso	Nome	Valor / observação
Região	—	`us-east-1` (N. Virginia) em **todos** os passos
VPC	VPC default	`172.31.0.0/16`, subnets públicas
Security groups	`sg-alb` · `sg-metabase` · `sg-rds`	regras na Parte I
RDS	`lab-metabase-db`	PostgreSQL 17 · `db.t4g.micro` · 20 GB · Free tier
Usuário master / banco inicial	`postgres` · `metabase`	senha self-managed — anote, vai no `.env`
Key pair	`lab-metabase-key`	RSA, `.pem`
EC2	`ec2-metabase`	Amazon Linux 2023 arm64 · `t4g.small` (2 GiB)
Pasta do projeto na EC2	`~/lab-metabase`	`docker-compose.yml`, `.env`, `seed-loja.sql`
Target group / ALB	`tg-metabase` · `alb-metabase`	HTTP:3000 · health `/api/health` · listener HTTP:80

💲 **Custos.** O RDS `db.t4g.micro` com 20 GB entra no free tier da conta. Fora dele: EC2 `t4g.small` ~US$ 0,017/h, ALB ~US$ 0,0225/h + uso, IPv4 público US$ 0,005/h. O lab de ~2 h fica em **centavos**, **desde que você faça a Limpeza** no mesmo dia — o ALB cobra mesmo sem tráfego.

📁 **Arquivos do lab.** Esta pasta do repositório traz `docker-compose.yml`, `.env.example` e `seed-loja.sql`. O conteúdo completo de cada um também está neste PDF (Parte IV), para colar direto na EC2 caso você não clone o repositório.

PARTE I · REDE E SEGURANÇA

I Três security groups encadeados

Criados primeiro, porque o RDS, a EC2 e o ALB vão pedir por eles

1 Confira a região **US East (N. Virginia) us-east-1** e acesse **EC2 → Network & Security → Security Groups → Create security group**. Crie os três abaixo, **nesta ordem** (o segundo referencia o primeiro, o terceiro referencia o segundo).

EC2 — N. Virginia

EC2 › Security Groups › Create security group

① sg-alb — porta de entrada pública

Security group name / Description	`sg-alb` · ALB do Metabase - HTTP publico
VPC	vpc-... (default) · 172.31.0.0/16
Inbound rule	`HTTP · TCP 80 · Source: Anywhere-IPv4 0.0.0.0/0`

② sg-metabase — a EC2

Security group name / Description	`sg-metabase` · EC2 Metabase
Inbound rule 1	`Custom TCP · 3000 · Source: Custom → sg-alb`
Inbound rule 2	`SSH · TCP 22 · Source: My IP`
Inbound rule 3 (opcional, teste)	`Custom TCP · 3000 · Source: My IP` remova no fim

③ sg-rds — o banco

Security group name / Description	`sg-rds` · RDS PostgreSQL do lab
Inbound rule	`PostgreSQL · TCP 5432 · Source: Custom → sg-metabase`
Outbound (os três)	All traffic · 0.0.0.0/0 (padrão)



A origem de cada regra é o **security group** anterior, não um IP: qualquer instância com aquele SG passa, e nenhuma outra.

Figura 2 — Encadeamento de security groups. O banco não fica exposto nem para o seu notebook.

✅ **Checkpoint I:** três SGs na VPC default · `sg-alb` com 80 aberto · `sg-metabase` com 3000 vindo de `sg-alb` e 22 do seu IP · `sg-rds` com 5432 vindo de `sg-metabase`.

PARTE II · BANCO DE DADOS

II RDS PostgreSQL no free tier

Privado, sem redundância e com o banco interno do Metabase já criado

1 Acesse **Aurora and RDS → Databases → Create database** e preencha:

Aurora and RDS — N. Virginia	
Aurora and RDS › Databases › Create database	
Creation method	**Full configuration (Standard create)** · Easy create
Engine type / version	**PostgreSQL** · **PostgreSQL 17.x (padrão)**
Templates	Production · Dev/Test · **Free tier**
Availability and durability	**Single-AZ DB instance deployment (1 instance)**
DB instance identifier	`lab-metabase-db`
Master username	`postgres`
Credentials management	Managed in AWS Secrets Manager · **Self managed** · senha forte, **anote**
DB instance class	**Burstable · db.t4g.micro** (ou db.t3.micro) · 2 vCPU · 1 GiB
Storage	**General Purpose SSD · 20 GiB**
Storage autoscaling	☐ **desmarcado** fica nos 20 GB do free tier
Compute resource	**Don't connect to an EC2 compute resource**
VPC / DB subnet group	**Default VPC** · **default**
Public access	Yes · **No** só a EC2 fala com o banco
VPC security group	**Choose existing** → `sg-rds` (remova o "default")
Database authentication	**Password authentication**
Additional configuration → Initial database name	`metabase`
Backup retention / Encryption	**1 day** · ☑ **Enable encryption (aws/rds)**
Deletion protection	☐ **desmarcado** para a limpeza

2 Clique em **Create database**. Leva de 5 a 10 minutos até **Available** — **adiante a Parte III enquanto isso**. Depois, em **lab-metabase-db → Connectivity & security**, anote o **Endpoint** (`lab-metabase-db.xxxx.us-east-1.rds.amazonaws.com`).

🎟️ O que é "free tier" aqui

Para contas elegíveis, até 750 h/mês de `db.t4g.micro`/`db.t3.micro` Single-AZ, 20 GB de storage e 20 GB de backup. O template **Free tier** já trava as opções que gerariam cobrança (Multi-AZ, classes maiores). Confira o selo no console: as regras dependem da data de criação da conta.

🔐 TLS obrigatório

No RDS PostgreSQL 15+ o parâmetro `rds.force_ssl` vem ligado: conexões sem TLS são recusadas. Por isso o `docker-compose.yml` usa `sslmode=require` no Metabase e no seed.

✅ **Checkpoint II:** `lab-metabase-db` **Available** · `db.t4g.micro` · Single-AZ · Public access **No** · security group `sg-rds` · endpoint e senha anotados.

PARTE III · COMPUTAÇÃO

III EC2 com Amazon Linux 2023

A máquina que vai rodar o Docker — Graviton, 2 GiB de memória, IP público da VPC default

1 Acesse **EC2 → Instances → Launch instances** e preencha:

EC2 — N. Virginia

EC2 › Instances › Launch an instance

Name	`ec2-metabase`
AMI	Amazon Linux 2023 AMI · 64-bit (Arm)
Instance type	t4g.small · 2 vCPU · 2 GiB · Graviton (ou t3.small com AMI 64-bit x86)
Key pair	Create new key pair · `lab-metabase-key` · RSA · .pem
Network settings → Edit · VPC	vpc-... (default)
Subnet	subnet default em us-east-1a
Auto-assign public IP	Enable
Firewall (security groups)	Select existing security group → `sg-metabase`
Configure storage	20 GiB gp3 · imagens Docker ocupam ~1 GB

🧠 **Por que não a t4g.micro / t3.micro?** O Metabase roda na JVM e precisa de ~1 GB só para ele. Com 1 GiB de RAM a instância trava ou o container reinicia em loop. Com **2 GiB** (t4g.small/t3.small) e `-Xmx1g` ele roda folgado. Contas no plano gratuito novo costumam ter a `t4g.small/t3.small` com o selo *Free tier eligible* — confira no seletor de tipo.

2 Clique em **Launch instance**, espere **Running** e **3/3 checks passed**. Anote o **Public IPv4 address**.

3 **Primeiro acesso SSH** (usuário padrão do Amazon Linux: `ec2-user`):

Linux / macOS

```
$ chmod 400 ~/Downloads/lab-metabase-key.pem
$ ssh -i ~/Downloads/lab-metabase-key.pem \
    ec2-user@<IP_PUBLICO_EC2>
```

Windows (PowerShell)

```
PS> icacls .\lab-metabase-key.pem /inheritance:r
`
    /grant:r "$($env:USERNAME):(R)"
PS> ssh -i .\lab-metabase-key.pem ec2-user@<IP>
```

💡 **Sem SSH no seu computador?** Selecione a instância → **Connect → EC2 Instance Connect** abre o terminal no navegador. Ele exige a porta 22 liberada também para a prefix list `com.amazonaws.us-east-1.ec2-instance-connect` no `sg-metabase`.

✅ **Checkpoint III:** prompt `[ec2-user@ip-172-31-… ~]$` na `ec2-metabase` · `uname -m` responde `aarch64` (Graviton).

PARTE IV · CONTAINERS

IV Docker, docker-compose e o Metabase no ar

Instalar, criar o .env com as credenciais do RDS, carregar os dados e subir

1 **Instale o Docker e o plugin Compose** (o Amazon Linux 2023 traz o Docker no repositório, mas não o Compose):

ec2-metabase

```
$ sudo dnf install -y docker
$ sudo systemctl enable --now docker
$ sudo usermod -aG docker ec2-user        # permite usar docker sem sudo
$ sudo mkdir -p /usr/local/lib/docker/cli-plugins
$ sudo curl -sSL "https://github.com/docker/compose/releases/latest/download/docker-compose-
linux-$(uname -m)" \
    -o /usr/local/lib/docker/cli-plugins/docker-compose
$ sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
$ exit                                     # saia e entre de novo no SSH para o grupo docker valer
```

ec2-metabase (nova sessão SSH)

```
$ docker version --format "{{.Server.Version}}" && docker compose version
25.0.x
Docker Compose version v5.x.x
```

2 **Traga os arquivos do lab** para `~/lab-metabase`. Escolha um caminho:

A · Clonar o repositório

```
sudo dnf install -y git
git clone <url-do-repo> repo
cp -r repo/ambiente-laboratorio-dia-04-10 ~/lab-metabase
```

B · Colar o conteúdo

```
mkdir ~/lab-metabase && cd ~/lab-metabase
```

Para cada arquivo: `nano <nome>`, cole o conteúdo do **Anexo** deste PDF, **Ctrl+O**, **Enter**, **Ctrl+X**.

3 **Configure o `.env` com as credenciais do RDS** criado na Parte II — é daqui que o `docker-compose.yml` lê host, usuário e senha:

ec2-metabase · ~/lab-metabase

```
$ cd ~/lab-metabase && cp .env.example .env && chmod 600 .env
$ nano .env
# troque DB_HOST pelo endpoint do RDS, DB_PASSWORD pela senha master
# e (opcional) SITE_URL pelo DNS do ALB depois da Parte V
$ grep -v PASSWORD .env                    # confira sem mostrar as senhas
```

4 **Carregue os dados de exemplo** (uma única vez). O serviço `seed` conecta no RDS com TLS, cria o banco `loja` com 20 mil vendas e um usuário somente leitura:

ec2-metabase · ~/lab-metabase

```
$ docker compose --profile seed run --rm seed
CREATE DATABASE
DROP TABLE ... CREATE TABLE ... INSERT 0 20000 ... GRANT
  tabela  | count
----------+-------
 lojas    |    12
 produtos |    15
 vendas   | 20000
Seed concluido: bancos 'metabase' e 'loja' prontos.
```

🔌 **Travou em "connection timed out"?** É rede, não senha: confira se o RDS usa o `sg-rds` e se a regra 5432 tem como origem o `sg-metabase` (Parte I). Erro de **password authentication failed** é o `.env`.

5 **Suba o Metabase** e acompanhe a inicialização (na primeira vez ele cria ~170 tabelas no banco `metabase` do RDS e leva 1–3 min):

ec2-metabase · ~/lab-metabase

```
$ docker compose up -d
$ docker compose logs -f metabase          # Ctrl+C para sair do log
... Successfully verified PostgreSQL 17.x application database connection. ✅
... Metabase Initialization COMPLETE
$ docker compose ps
NAME       IMAGE                          STATUS                   PORTS
metabase   metabase/metabase:v0.63.19.1   Up 2 minutes (healthy)   0.0.0.0:3000->3000/tcp
$ curl -s localhost:3000/api/health
{"status":"ok"}
```

🗄️ Sem volume

O compose não monta disco: tudo o que o Metabase grava vai para o RDS. Pode apagar o container sem medo (a Parte VII prova isso).

🧠 -Xmx1g

Limita a heap da JVM a 1 GB e deixa o resto da `t4g.small` para o sistema e o Docker.

🔁 restart

`unless-stopped` religa o container se a EC2 reiniciar (o serviço docker está com `enable`).

✅ **Checkpoint IV:** seed mostrou 12 / 15 / 20000 · `docker compose ps` com **healthy** · `/api/health` responde ok. (Opcional: se liberou 3000 para o seu IP, `http://<IP_PUBLICO_EC2>:3000` já abre.)

PARTE V · PUBLICAÇÃO

V Application Load Balancer na frente da EC2

Target group na porta 3000 com health check, ALB público na porta 80

1 **Target group.** Acesse **EC2 → Load Balancing → Target Groups → Create target group**:

EC2 — N. Virginia

EC2 › Target groups › Create target group

Target type	**Instances** · IP addresses · Lambda
Target group name	`tg-metabase`
Protocol : Port · VPC	**HTTP** : `3000` · **default** · **HTTP1**
Health check path · Advanced	`/api/health` · healthy threshold `2` · interval `15` s
Register targets	marque `ec2-metabase` · porta `3000` → **Include as pending below**

2 **Load balancer.** **Load Balancers → Create load balancer → Application Load Balancer → Create**:

EC2 — N. Virginia

EC2 › Load balancers › Create Application Load Balancer

Load balancer name	`alb-metabase`
Scheme / IP type / VPC	**Internet-facing** · **IPv4** · **default**
Availability Zones and subnets	**us-east-1a** · **us-east-1b** mínimo 2 AZs; uma delas a da EC2
Security groups	`sg-alb` remova o "default"
Listeners and routing	**HTTP : 80** → Forward to `tg-metabase`

3 Clique em **Create load balancer**. Em ~3 min o estado vai de *Provisioning* para **Active**. Em **Target groups → tg-metabase → Targets** a instância deve ficar **Healthy**. Copie o **DNS name** do ALB e abra no navegador: `http://alb-metabase-123456789.us-east-1.elb.amazonaws.com`.

4 **Informe a URL pública ao Metabase** (usada em links de e-mail e compartilhamento) e recrie o container:

ec2-metabase · ~/lab-metabase

```
$ sed -i 's|^SITE_URL=.*|SITE_URL=http://<DNS_DO_ALB>|' .env
$ docker compose up -d                          # recria só o que mudou
```

Se você criou a regra opcional `3000 ← My IP` no `sg-metabase`, **remova agora**: a partir daqui o único caminho até o Metabase é o ALB.

🩺 **Por que /api/health?** O Metabase leva 1–3 min para iniciar. O `/api/health` responde `503` durante a inicialização e só dá `200 {"status":"ok"}` quando a aplicação e a conexão com o RDS estão de pé. O ALB só manda usuário para alvo realmente saudável. **HTTPS** fica como desafio: certificado no ACM + listener 443 (exige domínio próprio).

✅ **Checkpoint V:** `alb-metabase` **Active** em 2 AZs · `tg-metabase` com o alvo **Healthy** · o DNS do ALB abre a tela de boas-vindas do Metabase.

PARTE VI · USANDO O BI

VI Configurar o Metabase e montar um dashboard

Conta de admin, conexão com o banco loja e as primeiras perguntas

1. **Assistente inicial.** Pelo DNS do ALB, clique em **Let's get started** e escolha o idioma **Português (Brasil)**. Crie a conta de administrador (nome, e-mail, senha forte) — ela fica gravada no banco `metabase` do RDS.

2. **Adicione os dados.** Na etapa de dados (ou depois em **⚙️ Admin → Bancos de dados → Adicionar banco de dados**):

Metabase — alb-metabase

Admin › Bancos de dados › Adicionar banco de dados

Tipo de banco de dados	PostgreSQL
Nome de exibição	`Loja (RDS)`
Host	`lab-metabase-db.xxxx.us-east-1.rds.amazonaws.com`
Porta / Nome do banco	`5432` · `loja`
Nome de usuário / Senha	`metabase_leitura` · `valor de LEITURA_PASSWORD do .env`
Usar uma conexão segura (SSL)	☑️ **ligado** · Modo SSL **require**

🧩 **Dois bancos, dois papéis.** O banco `metabase` é o "cérebro" do BI (usuários, perguntas, dashboards) e é acessado pelo **master** via `.env`. O banco `loja` é o **dado analisado**, acessado pelo usuário `metabase_leitura`, que só tem `SELECT` — o BI não consegue apagar nada.

3. **Primeira pergunta sem SQL:** **+ Novo → Pergunta → Loja (RDS) → Vendas** · **Resumir**: *Soma de Valor Total* · **por**: *Loja → UF* · **Visualizar** · troque para gráfico de **Barras** · **Salvar** como **Faturamento por UF**.

4. **Uma pergunta em SQL:** **+ Novo → Consulta SQL → Loja (RDS)**:

Metabase · editor SQL

```sql
SELECT date_trunc('month', v.data_venda) AS mes,
       v.canal,
       sum(v.valor_total)                AS faturamento
FROM   vendas v
GROUP BY 1, 2
ORDER BY 1;
```

Visualize como **Linha** (eixo X = `mes`, série = `canal`) e salve como **Faturamento mensal por canal**.

5 **Monte o dashboard** `+ Novo → Painel` → nome **Painel de Vendas · Lab 04/10**, e adicione os cards:

Card	Visualização	Como criar
Faturamento total	Número	Vendas · Soma de Valor Total (sem agrupamento)
Faturamento por UF	Barras	pergunta salva no passo 3
Faturamento mensal por canal	Linha	pergunta SQL do passo 4
Top 5 produtos	Barras horizontais	Vendas · Soma de Quantidade por Produto → Nome · ordenar desc · limite 5
Vendas por categoria	Pizza	Vendas · Contagem por Produto → Categoria

Adicione um **filtro** de painel do tipo *Data* ligado a `data_venda` em todos os cards e um de *Texto* ligado a `Loja → Região`. Salve.

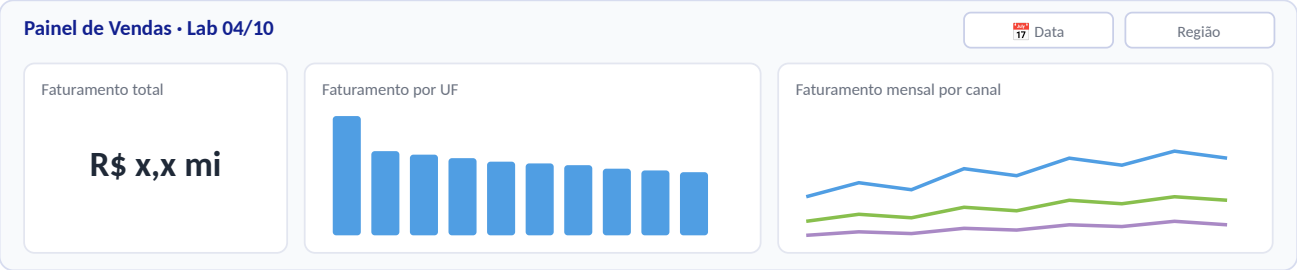


Figura 3 — Esboço do painel esperado (ilustrativo; os valores saem do seed aleatório de cada um).

✅ **Checkpoint VI:** conta admin criada · banco **Loja (RDS)** conectado com SSL e usuário `metabase_leitura` · 2 perguntas salvas · painel com 5 cards e 2 filtros, acessado pelo DNS do ALB.

PARTE VII · PROVAS E DESAFIOS

VII Onde os dados moram, de verdade

Container descartável, estado no RDS

1 **Destrua o container e suba de novo.** Seu login e o painel devem continuar lá:

ec2-metabase · ~/lab-metabase

```
$ docker compose down                       # remove o container (não há volume)
 Container metabase Removed
$ docker compose up -d && docker compose ps   # aguarde "healthy" (~1 min)
```

Recarregue o DNS do ALB: o Metabase pede login (não o assistente inicial) e o **Painel de Vendas** está intacto. Durante o intervalo, o ALB mostra **502/503** — é o health check tirando o alvo do ar.

2 **Olhe dentro do banco interno do Metabase**, direto da EC2, com um container descartável do cliente `psql`:

ec2-metabase · ~/lab-metabase

```
$ docker run --rm --env-file .env postgres:17-alpine sh -c \
  'PGPASSWORD=$DB_PASSWORD psql "host=$DB_HOST dbname=$DB_NAME user=$DB_USER sslmode=require" \
   -c "select email, date_joined::date from core_user;" \
   -c "select name from report_dashboard;"'
      email          | date_joined
----------------------+-------------
 voce@exemplo.com     | 2026-10-04
          name
--------------------------------
 Painel de Vendas · Lab 04/10
 ...                                        (+ exemplos que o Metabase cria)
```

🧪 Desafio 1 · Alta disponibilidade

Crie uma **segunda EC2** em `us-east-1b` com o mesmo `sg-metabase`, repita a Parte IV **sem o seed** (mesmo `.env`) e registre-a no `tg-metabase`. As duas usam o mesmo RDS: pare o container de uma e veja o ALB seguir respondendo pela outra.

🧪 Desafio 2 · Atualização sem perder nada

Troque a tag da imagem no `docker-compose.yml` por uma versão mais nova e rode `docker compose pull && docker compose up -d`. O Metabase migra o schema do banco `metabase` sozinho. Antes, tire um **snapshot manual** do RDS (rollback seguro).

✅ **Checkpoint VII:** após down/up o painel continua · `core_user` e `report_dashboard` mostram seus dados no RDS.

QUANDO ALGO NÃO FUNCIONA

Troubleshooting

Siga o pacote: ALB → target group → container → RDS

Sintoma	Causa provável	Como resolver
Seed: `connection timed out`	`sg-rds` sem 5432 vindo do `sg-metabase`; RDS com outro SG	RDS → Modify → Security group `sg-rds`; confira a regra de entrada
`password authentication failed`	senha ou usuário errados no `.env`	`nano .env`; senha master do RDS; sem aspas nem espaços
`no pg_hba.conf entry … no encryption`	conexão sem TLS	mantenha `sslmode=require` no compose e SSL ligado no Metabase
`could not translate host name`	`DB_HOST` errado ou RDS ainda criando	copie o endpoint de novo; aguarde **Available**
Container reinicia em loop / `Exited (137)`	falta de memória (instância de 1 GiB)	use `t4g.small`/`t3.small`; `docker compose logs metabase`
`permission denied … docker.sock`	grupo docker ainda não aplicado	saia do SSH e entre de novo (ou use `sudo`)
`docker: 'compose' is not a docker command`	plugin não instalado	refaça os comandos de `cli-plugins` da Parte IV
Target **Unhealthy**	Metabase ainda iniciando; porta/caminho errados; `sg-metabase` sem 3000 do `sg-alb`	aguarde o *healthy* no `docker compose ps`; health check `/api/health` na porta 3000
DNS do ALB não abre / 504	`sg-alb` sem 80; alvo não registrado	regra 80 de 0.0.0.0/0; registre a EC2 no `tg-metabase`
Metabase: erro ao adicionar **Loja (RDS)**	usuário/senha de leitura ou SSL desligado	`metabase_leitura` + `LEITURA_PASSWORD`; SSL ligado, modo require
Assistente inicial aparece de novo	Metabase apontando para outro banco (H2 local)	confira `MB_DB_*` no compose e o `.env`; veja o log "application database"

OBRIGATÓRIO NO FIM DO LAB

Limpeza dos recursos

A ordem importa: de fora para dentro

1. **Load balancer** `alb-metabase` → Actions → Delete. *Ele cobra por hora mesmo parado.*
2. **Target group** `tg-metabase` → Delete.
3. **Instância** `ec2-metabase` (e a do Desafio 1) → Instance state → **Terminate**.
4. **RDS** `lab-metabase-db` → Actions → Delete → desmarque *Create final snapshot* e *Retain automated backups* → digite `delete me`.
5. Snapshots manuais do Desafio 2: **RDS → Snapshots → Delete**.
6. **Security groups**, nesta ordem: `sg-rds`, `sg-metabase`, `sg-alb` (cada um depende do anterior).
7. **Key pair** `lab-metabase-key` → Delete. **Não apague a VPC default.**
8. Confira **Billing → Bills** e que não sobrou nada em **EC2**, **Load Balancers** e **RDS** em us-east-1.

⏱️ Se o `sg-metabase` não apagar ("has a dependent object"), é porque a instância ainda está terminando ou o `sg-rds` ainda referencia ele. Aguarde e siga a ordem do passo 6.

Checklist de evidências para entrega

- ☐ Três SGs com o encadeamento da Figura 2
- ☐ RDS `db.t4g.micro` · Public access **No**
- ☐ `docker compose ps` com **healthy**
- ☐ Saída do seed (12 / 15 / 20000)
- ☐ Target group com alvo **Healthy**
- ☐ Navegador abrindo o DNS do ALB
- ☐ Painel com 5 cards e 2 filtros
- ☐ Prova do down/up (Parte VII)
- ☐ `core_user` consultado no RDS
- ☐ Print final da limpeza (nada rodando)

Perguntas para discussão

Arquitetura

Por que guardar o estado do Metabase no RDS em vez de num volume Docker na EC2? O que isso permite fazer com a EC2?

Segurança

O que mudaria para tirar o IP público da EC2 e ainda conseguir administrá-la? (dica: subnet privada + NAT + Session Manager)

Custo

Quais recursos deste lab cobram mesmo sem nenhum acesso? Qual deles sai do free tier primeiro?

Anexo A · docker-compose.yml

~/lab-metabase/docker-compose.yml

```
# Laboratório 04/10 - Metabase na EC2 usando o RDS PostgreSQL (free tier) como banco
# Uso:
#   1) cp .env.example .env   e preencha com os dados do SEU RDS
#   2) docker compose --profile seed run --rm seed   (uma vez: cria o banco "loja" com dados de exemplo)
#   3) docker compose up -d
#   4) acesse pelo DNS do Load Balancer (porta 80) ou http://<ip-da-ec2>:3000 para teste

services:
  metabase:
    image: metabase/metabase:v0.63.19.1
    container_name: metabase
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      # Banco interno do Metabase (usuarios, perguntas, dashboards) fica no RDS
      MB_DB_TYPE: postgres
      MB_DB_CONNECTION_URI: "postgres://${DB_HOST}:5432/${DB_NAME}?sslmode=require"
      MB_DB_USER: ${DB_USER}
      MB_DB_PASS: ${DB_PASSWORD}
      # Memoria da JVM: cabe numa instancia de 2 GiB (t4g.small / t3.small)
      JAVA_OPTS: "-Xmx1g"
      JAVA_TIMEZONE: America/Fortaleza
      MB_SITE_URL: ${SITE_URL:-}
    healthcheck:
      test: ["CMD-SHELL", "curl -fs http://localhost:3000/api/health || exit 1"]
      interval: 30s
      timeout: 5s
      retries: 10
      start_period: 120s

  # Servico de uso unico: cria os bancos "metabase" e "loja" e carrega dados de exemplo
  seed:
    image: postgres:17-alpine
    profiles: ["seed"]
    environment:
      PGHOST: ${DB_HOST}
      PGUSER: ${DB_USER}
      PGPASSWORD: ${DB_PASSWORD}
      PGSSLMODE: require
      APP_DB: ${DB_NAME}
      LEITURA_PASSWORD: ${LEITURA_PASSWORD}
    volumes:
      - ./seed-loja.sql:/seed/seed-loja.sql:ro
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        set -e
        psql -d postgres -v ON_ERROR_STOP=1 -tc "SELECT 1 FROM pg_database WHERE datname='$$APP_DB'" |
grep -q 1 \
          || psql -d postgres -v ON_ERROR_STOP=1 -c "CREATE DATABASE \"$$APP_DB\""
        psql -d postgres -v ON_ERROR_STOP=1 -tc "SELECT 1 FROM pg_database WHERE datname='loja'" | grep
-q 1 \
          || psql -d postgres -v ON_ERROR_STOP=1 -c "CREATE DATABASE loja"
        psql -d loja -v ON_ERROR_STOP=1 -v leitura_pw="$$LEITURA_PASSWORD" -f /seed/seed-loja.sql
        echo "Seed concluido: bancos '$$APP_DB' e 'loja' prontos."
```

Anexo B · .env.example

~/lab-metabase/.env.example → copie para .env

```
# Copie para .env e preencha com os dados do SEU RDS (nao versione o .env!)
# Endpoint em: RDS > Databases > lab-metabase-db > Connectivity & security > Endpoint
DB_HOST=lab-metabase-db.xxxxxxxxxxx.us-east-1.rds.amazonaws.com
DB_USER=postgres
DB_PASSWORD=TroqueEstaSenha123
# Banco interno do Metabase (criado pelo seed ou pelo "Initial database name" do RDS)
DB_NAME=metabase
# Senha do usuario somente-leitura que o Metabase usa para consultar o banco "loja"
LEITURA_PASSWORD=LeituraLab2026
# Opcional: DNS do Load Balancer, ex.: http://alb-metabase-123456.us-east-1.elb.amazonaws.com
SITE_URL=
```

Anexo C · seed-loja.sql

~/lab-metabase/seed-loja.sql

```
-- Dados de exemplo para o Metabase: rede de lojas fictícia (Lab 04/10)
-- Executado pelo servico "seed" do docker-compose dentro do banco "loja".
DROP TABLE IF EXISTS vendas, produtos, lojas;

CREATE TABLE lojas (
  id      serial PRIMARY KEY,
  nome    text NOT NULL,
  cidade  text NOT NULL,
  uf      char(2) NOT NULL,
  regiao  text NOT NULL
);

CREATE TABLE produtos (
  id         serial PRIMARY KEY,
  nome       text NOT NULL,
  categoria  text NOT NULL,
  preco      numeric(10,2) NOT NULL
);

CREATE TABLE vendas (
  id          bigserial PRIMARY KEY,
  data_venda  date NOT NULL,
  loja_id     int NOT NULL REFERENCES lojas(id),
  produto_id  int NOT NULL REFERENCES produtos(id),
  quantidade  int NOT NULL,
  valor_total numeric(12,2) NOT NULL,
  canal       text NOT NULL
);

INSERT INTO lojas (nome, cidade, uf, regiao) VALUES
 ('Loja Aldeota','Fortaleza','CE','Nordeste'), ('Loja Centro','Fortaleza','CE','Nordeste'),
 ('Loja Boa Viagem','Recife','PE','Nordeste'), ('Loja Pituba','Salvador','BA','Nordeste'),
 ('Loja Ponta Negra','Natal','RN','Nordeste'), ('Loja Paulista','São Paulo','SP','Sudeste'),
 ('Loja Copacabana','Rio de Janeiro','RJ','Sudeste'), ('Loja Savassi','Belo Horizonte','MG','Sudeste'),
 ('Loja Asa Sul','Brasília','DF','Centro-Oeste'), ('Loja Moinhos','Porto Alegre','RS','Sul'),
 ('Loja Batel','Curitiba','PR','Sul'), ('Loja Umarizal','Belém','PA','Norte');

INSERT INTO produtos (nome, categoria, preco) VALUES
 ('Protetor solar FPS 50','Dermocosméticos',79.90), ('Hidratante facial','Dermocosméticos',59.90),
 ('Shampoo anticaspa','Higiene',32.50), ('Creme dental','Higiene',8.90),
 ('Escova dental','Higiene',12.90), ('Vitamina C 1g','Suplementos',29.90),
 ('Whey protein 900g','Suplementos',149.90), ('Ômega 3','Suplementos',54.90),
 ('Fralda infantil G','Infantil',89.90), ('Lenço umedecido','Infantil',14.90),
 ('Termômetro digital','Saúde',24.90), ('Medidor de pressão','Saúde',189.90),
 ('Álcool em gel','Higiene',9.90), ('Repelente','Saúde',27.90), ('Colágeno','Suplementos',99.90);

-- ~20 mil vendas espalhadas pelos últimos 12 meses
INSERT INTO vendas (data_venda, loja_id, produto_id, quantidade, valor_total, canal)
SELECT d, l, p, q, round((q * pr.preco * (0.9 + random()*0.2))::numeric, 2), c
FROM (
  SELECT (current_date - (random()*365)::int)                          AS d,
         1 + (random()*11)::int                                         AS l,
         1 + (random()*14)::int                                         AS p,
         1 + (random()*4)::int                                          AS q,
         (ARRAY['Loja física','App','Site','Televendas'])[1 + (random()*3)::int] AS c
  FROM generate_series(1, 20000)
```

```
) s JOIN produtos pr ON pr.id = s.p;

CREATE INDEX ON vendas (data_venda);
CREATE INDEX ON vendas (loja_id);

-- Usuário somente leitura para o Metabase consultar (boa prática: não usar o master)
DO $$ BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'metabase_leitura') THEN
    CREATE ROLE metabase_leitura LOGIN;
  END IF;
END $$;
ALTER ROLE metabase_leitura PASSWORD :'leitura_pw';
GRANT CONNECT ON DATABASE loja TO metabase_leitura;
GRANT USAGE ON SCHEMA public TO metabase_leitura;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO metabase_leitura;

SELECT 'lojas' AS tabela, count(*) FROM lojas
UNION ALL SELECT 'produtos', count(*) FROM produtos
UNION ALL SELECT 'vendas', count(*) FROM vendas;
```